

A. Project Overview

Goal: The primary goal of this project was to analyze the web-Google.txt dataset using Breadth-First Search (BFS) and degree centrality to uncover structural insights into the graph of web pages. BFS was employed to find the shortest paths between nodes, and degree centrality was used to identify the most influential nodes in terms of connectivity.

Dataset: The dataset used in this project is the web-Google.txt file sourced from SNAP (Stanford Network Analysis Project). It represents a graph of web pages with directed edges indicating hyperlinks between them. The dataset is too large for GitHub (67 MB), so it was stored locally within the project directory for processing.

B. Data Processing

Loading the Dataset into Rust:

The dataset was loaded into the program by converting the .txt file to .csv format using the csv_converter.rs module. This module reads the edges from the .txt file, where each line represents a connection between two nodes, and writes them into a .csv file with columns for FromNodeId and ToNodeId.

Once the dataset was in CSV format, the graph.rs module was used to parse the CSV file and build a graph. The build_graph_from_csv function constructs a HashMap<usize, Vec<usize>> representing the graph, where the key is a node (web page) and the value is a vector of outgoing edges (links to other web pages).

Data Cleaning and Transformations:

The conversion from .txt to .csv involved basic data cleaning to skip comment lines in the .txt file. The dataset was then transformed into a graph structure using Rust's standard library, making it ready for graph algorithms like BFS and centrality analysis.

C. Code Structure

Modules

→ main.rs

The main.rs module is the entry point of the program. It coordinates the flow of the project, starting from the conversion of the .txt file to CSV, building the graph, sampling a connected subgraph, and running BFS and degree centrality algorithms. It displays results at each stage of the analysis.

→ `csv_converter.rs`

This module handles the conversion of the raw `.txt` file into a CSV format. It reads the graph data from the `.txt` file, writes it to a `.csv` file, and skips comment lines.

→ `graph.rs`

This module builds the graph from the CSV file. The `build_graph_from_csv` function parses the CSV file and stores the nodes and edges in a `HashMap<usize, Vec<usize>>`. It ensures that each node and its neighbors are added to the graph properly.

→ `algorithms.rs`

This module implements the core graph algorithms used in the project. It contains two main functions:

- ◆ `bfs`: Implements the Breadth-First Search algorithm to find the shortest path between two nodes.
- ◆ `degree centrality`: Computes and prints the degree centrality of each node in the graph, highlighting the most connected nodes by their out-degree.

Key Functions & Types

→ Graph Struct

- ◆ Purpose: The graph is represented as a `HashMap<usize, Vec<usize>>`, where each node (web page) is mapped to a list of neighboring nodes (linked pages).
- ◆ Core Logic: The graph is populated by reading the CSV file, where each entry (edge) is added to the respective node's list of outgoing links.

→ BFS Algorithm

- ◆ Purpose: Finds the shortest path between two nodes in the graph.
- ◆ Core Logic: BFS explores the graph level by level starting from the source node. It uses a queue to visit neighbors and a parent map to reconstruct the path once the target node is reached.

→ Degree Centrality

- ◆ Purpose: Measures the connectivity of each node by counting its incoming and outgoing edges.
- ◆ Core Logic: The function calculates both in-degrees and out-degrees for each node and prints the top 10 nodes by out-degree.

Main Workflow

1. File Conversion: The `.txt` file is converted into `.csv` using `csv_converter.rs`.

2. Graph Construction: The graph is built using the `build_graph_from_csv` function in `graph.rs`.
3. Subgraph Sampling: A random seed node is selected, and BFS is performed to sample a connected subgraph.
4. Graph Analysis: BFS is used to find shortest paths, and degree centrality is calculated and displayed.
5. Results Display: The program prints the BFS paths between nodes and the degree centrality of the top 10 nodes.

D. Tests

```
PS C:\Users\Sanya Sajin\Downloads\DS210_Project> cargo test
Compiling DS210_Project v0.1.0 (C:\Users\Sanya Sajin\Downloads\DS210_Project)
Finished `test` profile [unoptimized + debuginfo] target(s) in 3.88s
Running unittests src\main.rs (target\debug\deps\DS210_Project-217a43290cdfcb4b.exe)

running 3 tests
test algorithms::tests::test_degree_centrality ... ok
test algorithms::tests::test_bfs ... ok
test csv_converter::tests::test_convert_txt_to_csv ... ok

test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.01s

PS C:\Users\Sanya Sajin\Downloads\DS210_Project> 
```

- Test `degree_centrality`:
 - ◆ What it checks: Ensures degree centrality is correctly calculated.
 - ◆ Why it matters: Verifies that the most connected nodes are identified.
- Test `bfs`:
 - ◆ What it checks: Verifies that BFS finds the shortest path between nodes.
 - ◆ Why it matters: Ensures BFS works for graph traversal.
- Test `convert_txt_to_csv`:
 - ◆ What it checks: Validates the conversion from `.txt` to `.csv`.
 - ◆ Why it matters: Ensures the dataset is properly formatted for processing.

E. Results

```

PS C:\Users\Sanya Sajin\Downloads\DS210_Project> cargo run --release
    Finished `release` profile [optimized] target(s) in 0.12s
    Running `target\release\DS210_Project.exe`
Full graph loaded: 875713 nodes
Connected subgraph from seed 304759: 175143 nodes

-- BFS from 304759 to targets --
Path 304759→554934: [304759, 721816, 25410, 610361, 136014, 622920, 286867, 277876, 429917, 745518, 554934]
Path 304759→108745: [304759, 721816, 25410, 610361, 136014, 622920, 286867, 555924, 49944, 25746, 108745]
Path 304759→711294: [304759, 721816, 25410, 610361, 136014, 622920, 286867, 324502, 250114, 28117, 711294]

-- Degree Centrality (top 10) --
Node 412410: In-degree = 2, Out-degree = 268
Node 600479: In-degree = 3, Out-degree = 265
Node 156950: In-degree = 196, Out-degree = 257
Node 885728: In-degree = 211, Out-degree = 256
Node 667584: In-degree = 9, Out-degree = 253
Node 685695: In-degree = 345, Out-degree = 248
Node 579314: In-degree = 37, Out-degree = 244
Node 838278: In-degree = 71, Out-degree = 225
Node 302733: In-degree = 78, Out-degree = 216
Node 915273: In-degree = 31, Out-degree = 213
PS C:\Users\Sanya Sajin\Downloads\DS210_Project>

```

Program Outputs:

The program processed a connected subgraph from the main graph, performing BFS to find the shortest paths between nodes and calculating degree centrality for the top 10 nodes. The analysis is based on a randomly sampled subgraph.

Interpretation:

The BFS results show the shortest paths within the sampled subgraph, indicating how nodes are interconnected in this subset of the graph. The degree centrality analysis identifies the most connected nodes by their in-degree and out-degree, providing insight into the node importance within the sampled portion of the graph.

F. Usage Instructions

Building and Running the Code:

Clone the Repository: git clone https://github.com/sanya-sajin/Google_Graph_Analysis

Install Dependencies: cargo build

Run the Program: cargo run --release

Command-Line Arguments/User Interaction: The program requires no command-line arguments. It runs automatically upon execution and processes the dataset.

Expected Runtime: The program takes approximately 20-30 seconds to process the sampled subgraph and perform the analysis.

